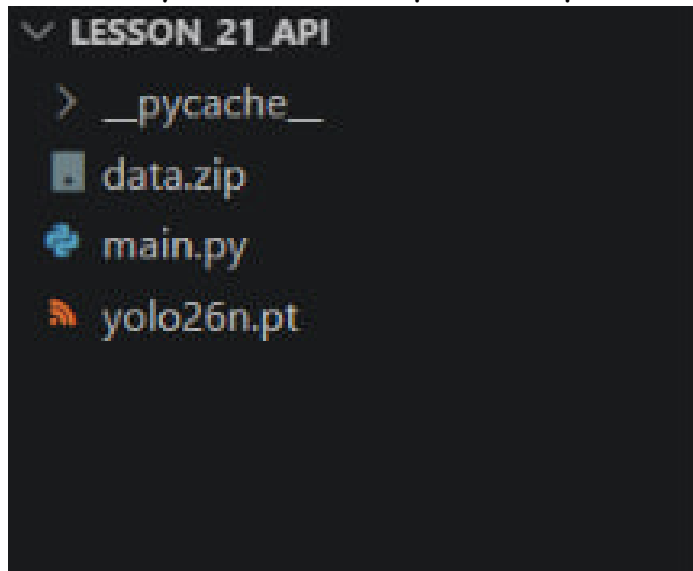
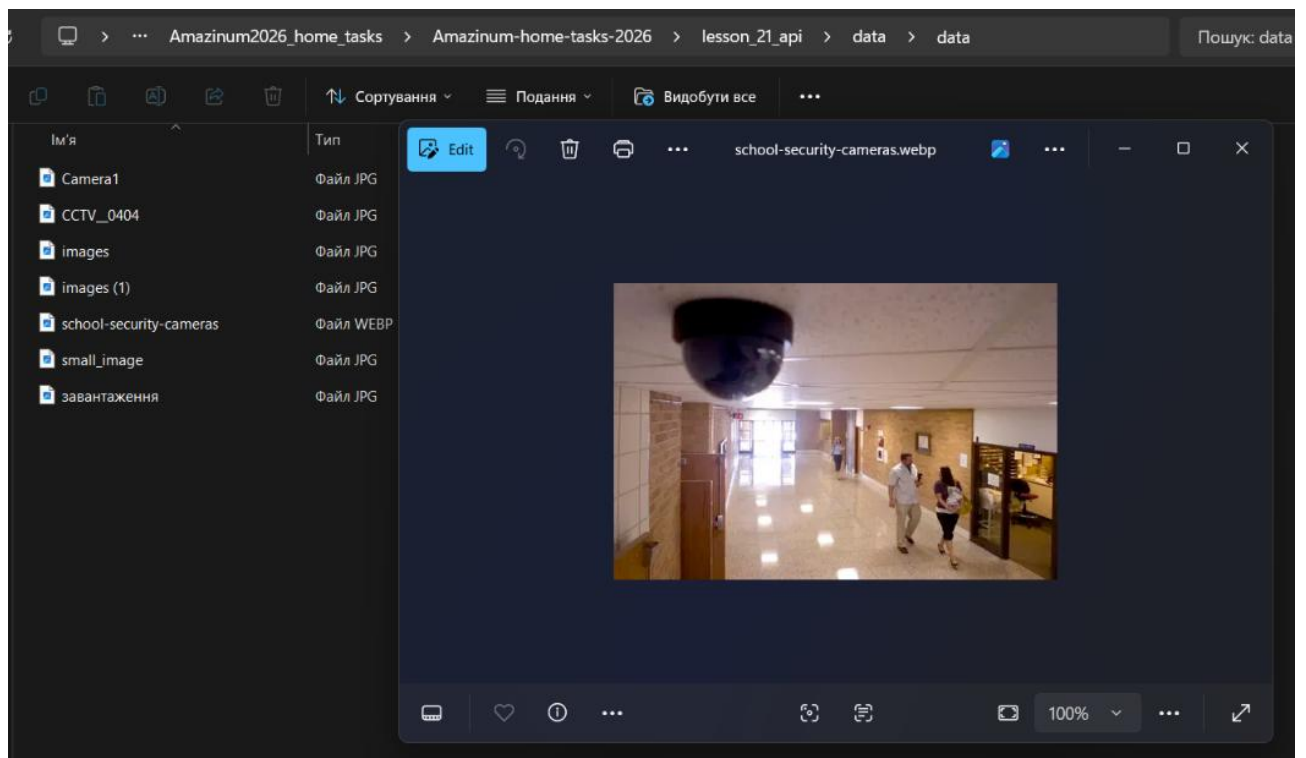


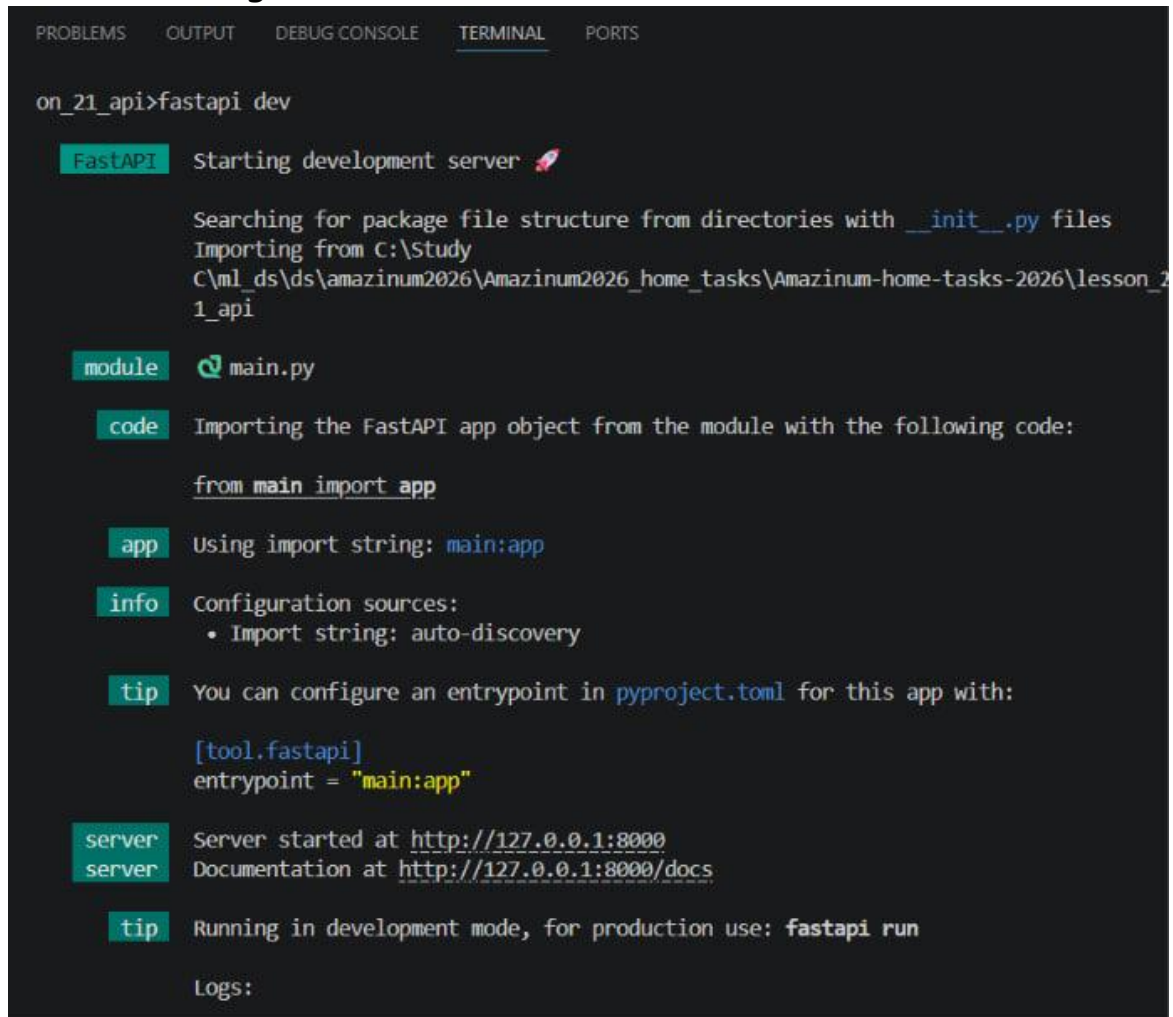
Directory with the script and input data archive:



Example image from data.zip archive:



Server running (FastAPI dev in terminal):



The screenshot shows a terminal window with the following content:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

on_21_api>fastapi dev

FastAPI Starting development server 🚀

Searching for package file structure from directories with __init__.py files
Importing from C:\Study
C:\ml_ds\ds\amazinum2026\Amazinum2026_home_tasks\Amazinum-home-tasks-2026\lesson_21_api

module 📄 main.py

code Importing the FastAPI app object from the module with the following code:

from main import app

app Using import string: main:app

info Configuration sources:
  • Import string: auto-discovery

tip You can configure an entrypoint in pyproject.toml for this app with:

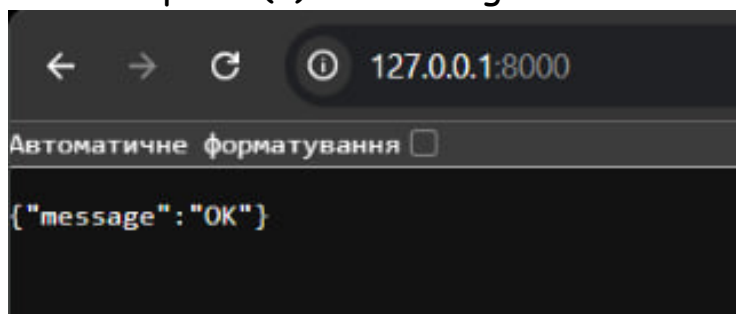
[tool.fastapi]
entrypoint = "main:app"

server Server started at http://127.0.0.1:8000
server Documentation at http://127.0.0.1:8000/docs

tip Running in development mode, for production use: fastapi run

Logs:
```

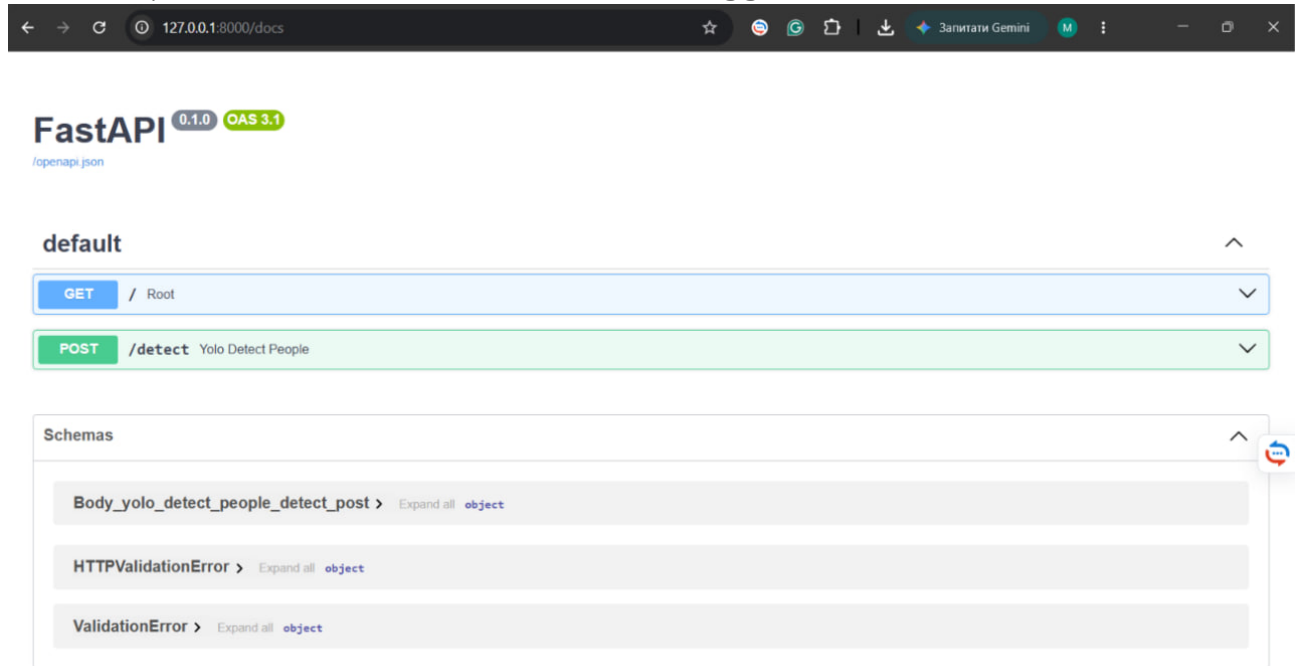
Root endpoint (/) confirming server is running:



The screenshot shows a web browser window with the address bar set to `127.0.0.1:8000`. The page content displays the JSON response from the root endpoint:

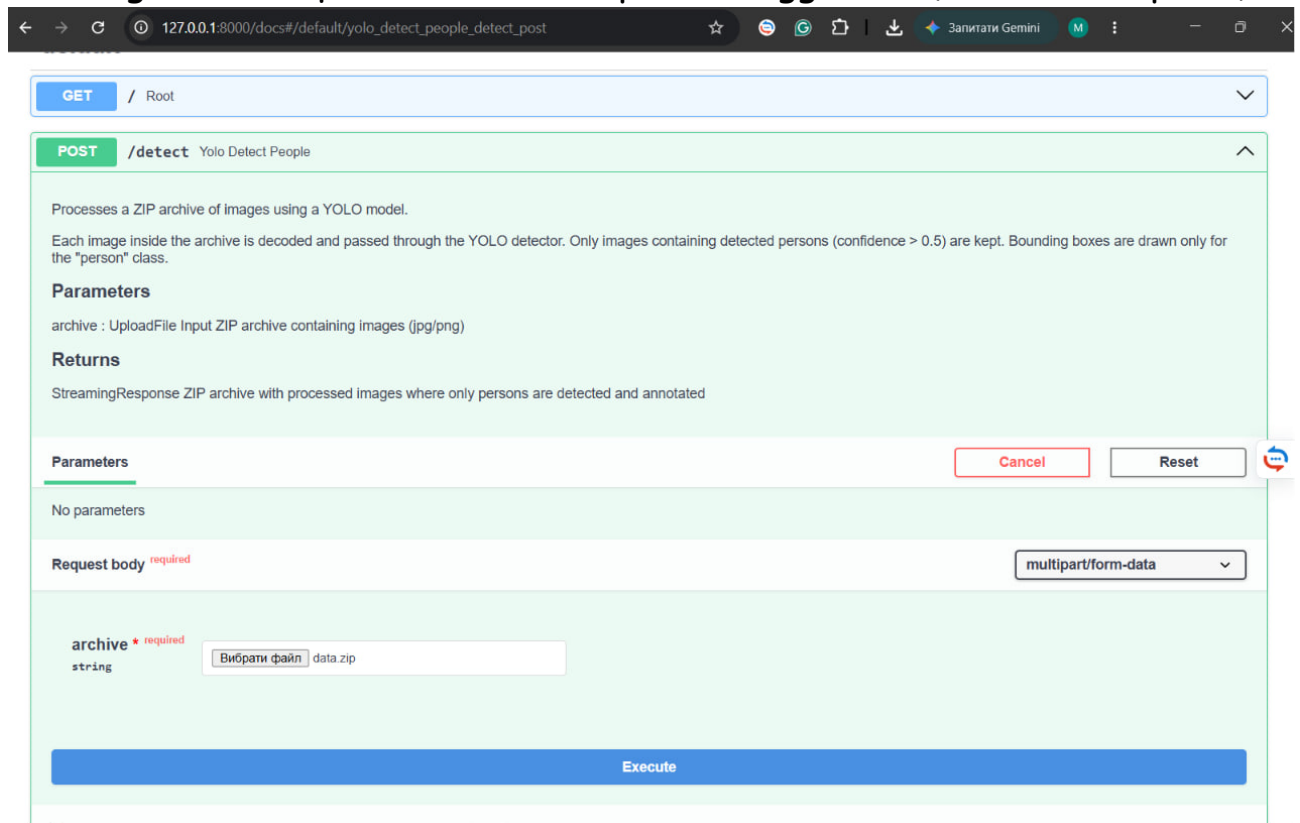
```
{ "message": "OK" }
```

API endpoints documentation (/docs Swagger UI):



The screenshot shows the Swagger UI for a FastAPI application. The top bar indicates the API version is 0.1.0 and the OpenAPI spec is 3.1. The main content area is titled "default" and lists two endpoints: a GET endpoint at the root and a POST endpoint at /detect. Below the endpoints, the "Schemas" section is expanded, showing three schemas: Body_yolo_detect_people_detect_post, HTTPValidationError, and ValidationError. Each schema is marked as an "object" type.

Sending POST request with data.zip via Swagger UI (/detect endpoint):



The screenshot shows the Swagger UI configuration for the POST endpoint at /detect. The endpoint is titled "Yolo Detect People". The description states: "Processes a ZIP archive of images using a YOLO model. Each image inside the archive is decoded and passed through the YOLO detector. Only images containing detected persons (confidence > 0.5) are kept. Bounding boxes are drawn only for the 'person' class." The "Parameters" section is empty. The "Returns" section indicates a "StreamingResponse ZIP archive with processed images where only persons are detected and annotated". The "Request body" section is marked as "required" and shows a "multipart/form-data" type. A file upload field is present with the label "archive" and a red asterisk indicating it is required. The file name is "data.zip". A blue "Execute" button is at the bottom.

Successful execution of request and download of processed.zip archive:

The screenshot displays a web application interface for testing an API endpoint. The 'Request body' section shows a file upload field labeled 'archive' with a 'Вибрати файл' (Choose file) button and a 'data.zip' file selected. An 'Execute' button is present. The 'Responses' section shows the curl command used for the request and the server response, which includes a 200 status code and a 'content-disposition: attachment; filename=processed.zip' header. A file explorer overlay shows the contents of the downloaded 'processed.zip' archive, including files like 'Camera1.jpg', 'images (1).jpg', 'lucid-origin_Empty_office_corridor_2042_9_30_AM_Camera_surveillance_unit_at_ceiling_corner_3-0.jpg', 'CCTV_0404.jpg', 'school-security-cameras.webp', and 'завантаження.jpg'.

Request body

archive * required
string

Вибрати файл data.zip

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/detect' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'archive=@data.zip;type=application/x-zip-compressed'
```

Request URL

http://127.0.0.1:8000/detect

Server response

Code Details

200

Response body

[Download file](#)

Response headers

```
content-disposition: attachment; filename=processed.zip
content-type: application/zip
date: Thu, 25 Jun 2026 17:21:08 GMT
server: uvicorn
transfer-encoding: chunked
```

Example of annotated image from processed.zip:

The screenshot shows a file explorer window with the path 'Завантаження > processed > data'. The 'school-security-cameras' folder is selected, and its contents are listed: 'school-security-cameras' (Файл WEBP), 'small_image' (Файл JPG), and 'завантаження' (Файл JPG). A preview window is open, showing an image of a hallway with two people walking. The image is annotated with bounding boxes and labels: 'person 0.99' and 'person 0.96'.

Завантаження > processed > data

school-security-cameras

school-security-cameras

small_image

завантаження

Файл WEBP

Файл JPG

Файл JPG

school-security...

person 0.99

person 0.96